

Instructions:

- We will only grade what is written in the space provided below each question
- Make sure to write your name and ERP on each page
- Extra sheets may be used for rough work but make sure to write your name and ERP on them. Answers on extra sheets will not be graded
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam
- In this exam, 1k means 1000 and not 1024. Similarly, 1M means 10^6 . Capital 'B' is used for bytes, while small 'b' is used for bits. Therefore, 1kb is 1000 bits, and 4MB is 4×10^6 bytes
- Only writing or copying formulas without appropriate final numerical calculations will not earn you any credit.
- It is okay to leave the trivial steps and write the important ones. However, writing just the final answer that is incorrect will not earn you any partial credit.
- Make sure you have all 9 pages (printed on 5 two-sided sheets) of the exam before you begin

Name: _____

Solution

ERP: _____

Q1. Merkle Tree (10 marks)

- a) Explain what a **Merkle Tree** is and describe the process of constructing a Merkle Tree from a set of transactions in a block (6 marks)
- b) Discuss the **time complexity** of verifying whether a specific transaction is included in a block in the following two cases:
 - i. When no Merkle Tree is used (2 marks)
 - ii. When a Merkle Tree is used (2 marks)

a)

A **Merkle Tree** is a binary tree used to efficiently summarize and verify the transactions in a block.

- The **leaf nodes** are hashes of individual transactions
- Each **internal node** is the hash of the concatenation of its two child nodes
- The **root hash**, called the **Merkle Root**, represents all transactions in the block and is stored in the block header

Construction:

1. Compute the hash of each transaction.
2. Pair adjacent hashes and hash their concatenation.
3. Repeat this process until a single hash (the Merkle Root) remains.

b)

Verification Complexity:

- **Without Merkle Tree:**
The verifier must scan all transactions in the block.
Time Complexity: $O(n)$
- **With Merkle Tree:**
Verification uses a **Merkle proof**, which contains hashes along the path from the transaction to the root. (Tree Height)
Time Complexity: $O(\log n)$

Q2. Simplified Bitcoin Transaction Signatures (45 marks)

In Bitcoin, transactions are digitally signed by the **payer** using their secret **private key**. Anyone can verify the signature using the **public key**.

For this question, we use the following simplified signature scheme.

Alice's keys:

- Public key ($p = 7$)
- Private key ($s = 3$)

Signature Generation

The signature is generated using the private key:

$$sig = h(m)^s \bmod 33$$

Signature Verification

To verify a signature:

$$sig^p \bmod 33 \stackrel{?}{=} h(m)$$

If both sides match, the signature is **valid**.

Transaction Format

Transactions are represented as a list:

payerID payeeID amount

Example:

1 4 10 means payer = 1, payee = 4, and amount = 10 coins

Hash Function

The hash function to be used is:

$$h(m) = \lfloor e^m + 100 \rfloor \bmod 32$$

Note: $\lfloor x \rfloor$ is the floor function, which gets rid of the decimal part of x
For E.g. $\lfloor 32.53 \rfloor$ becomes 32

For a list of numbers:

$$h(m_1 m_2 m_3) = h((m_1 + m_2 + m_3) \bmod 50)$$

a) Verify that Alice generated the following signed transactions:

- 1 4 13 26 where: 1 4 13 $\rightarrow$ transaction data and 26 $\rightarrow$ signature (10 marks)
- 1 3 9 12 where: 1 3 9 $\rightarrow$ transaction data and 12 $\rightarrow$ signature (10 marks)

b) Alice wants to send 5 coins to user 4, which results in the transaction data $\rightarrow$ 1 4 5, and you must construct the final signed transaction (10 marks)

a) i.

Step 1: Compute hash

Transaction data: 1 4 13

$$h(m) = h(1\ 4\ 13)$$

$$h(m) = h((1 + 4 + 13) \bmod 50)$$

$$h(m) = h(18 \bmod 50)$$

$$h(m) = h(18)$$

$$h(m) = \lfloor e^{18} + 100 \rfloor \bmod 32$$

$$h(m) = 650325906 \bmod 32$$

$$h(m) = 5$$

Step 2: Verify signature

Signature: 26

$$p = 7$$

$$\text{sig}^p \bmod 33 = 26^7 \bmod 33$$

$$\text{sig}^p \bmod 33 = 8,031,810,176 \bmod 33$$

$$\text{sig}^p \bmod 33 = 5$$

$$\text{sig}^p \bmod 33 = h(m) \quad \text{Both sides match; the signature is valid.}$$

Therefore, the transaction **was signed by Alice**.

a) ii.

Step 1: Compute hash

Transaction data: 1 3 9

$$h(m) = h(1\ 3\ 9)$$

$$h(m) = h((1 + 3 + 9) \bmod 50)$$

$$h(m) = h(13 \bmod 50)$$

$$h(m) = h(13)$$

$$h(m) = \lfloor e^{13} + 100 \rfloor \bmod 32$$

$$h(m) = 442513 \bmod 32$$

$$h(m) = 17$$

Step 2: Verify signature

Signature: 12

$$p = 7$$

$$\text{sig}^p \bmod 33 = 12^7 \bmod 33$$

$$\text{sig}^p \bmod 33 = 35,831,808 \bmod 33$$

$$\text{sig}^p \bmod 33 = 12$$

$$\text{sig}^p \bmod 33 \neq h(m) \quad \text{Both sides do not match; the signature is invalid.}$$

Therefore, the transaction **was NOT signed by Alice**.

b)

Step 1: Compute hash

Transaction data: 1 4 5

$$h(m) = h(1\ 4\ 5)$$

$$h(m) = h((1 + 4 + 5) \bmod 50)$$

$$h(m) = h(10 \bmod 50)$$

$$h(m) = h(10)$$

$$h(m) = \lfloor e^{10} + 100 \rfloor \bmod 32$$

$$h(m) = 22126 \bmod 32$$

$$h(m) = 14$$

Step 2: Generate signature

Hash: 14

$$s = 3$$

$$\text{sig} = h(m)^s \bmod 33$$

$$\text{sig} = 14^3 \bmod 33$$

$$\text{sig} = 2744 \bmod 33$$

$$\text{sig} = 5$$

Final transmitted transaction

1 4 5 5

c) Alice now wants to spend an output from a previous transaction following P2PKH.

The locking script (scriptPubKey) is:

OP_DUP OP_HASH [hash_of_Alice_public_key] OP_EQUALVERIFY OP_CHECKSIG

Use the same hashing function, and Alice's public key as given in the question, and the signature from part b)

- i. Compute the hash of Alice's public key and write the final **scriptPubKey** (5 marks)
- ii. Write the **scriptSig** that Alice provides (2 marks)
- iii. Combine the scripts and simulate the stack execution until the script returns **true**. You may assume that OP_CHECKSIG succeeds, and you do not need to provide the steps for signature verification (8 marks)

c) i.

Step 1: Compute hash

Public key: 7

$$h(p) = h(7)$$

$$h(p) = [e^7 + 100] \bmod 32$$

$$h(p) = 1196 \bmod 32$$

$$h(p) = 12$$

Step 2: Rewrite scriptPubKey

hash_of_Alice_public_key: 12

scriptPubKey: OP_DUP OP_HASH 12 OP_EQUALVERIFY OP_CHECKSIG

c) ii.

Signature: 5

Public key: 7

scriptSig: 5 7

c) iii.

Combined Script: 5 7 OP_DUP OP_HASH 12 OP_EQUALVERIFY OP_CHECKSIG

Stack Execution

Start

[]

Push 5

[5]

Push 7

[5, 7]

OP_DUP

[5, 7, 7]

OP_HASH

h(7)=12

[5, 7, 12]

Push 12

[5, 7, 12, 12]

OP_EQUALVERIFY

12 = 12

Stack:

[5, 7]

OP_CHECKSIG

Signature valid → push **true**

[true]

Final result:

Script evaluates to **true**.

Q3. Proof-of-Work and Mining (10 marks)

- a) What is the **target hash** value in Proof-of-Work blockchains? (3 marks)
- b) Explain how the target hash affects the process of finding a valid **nonce** or **block hash** (3 marks)
- c) What is the potential consequence when the target makes finding a nonce:
- i. Very easy (2 marks)
 - ii. Very difficult (2 marks)

a)

A **target hash value** is a threshold that the hash of a block must be **less than or equal to** for the block to be considered valid. It determines the **mining difficulty** in a Proof-of-Work blockchain.

b)

Miners repeatedly change the **nonce** in the block header and compute the block hash until:

$$\text{Hash}(\text{block header}) \leq \text{Target}$$

- A **higher target** → easier to find a valid hash (lower difficulty).
- A **lower target** → harder to find a valid hash (higher difficulty).

c) i. and ii.

Consequences:

- **If mining is too easy:**
Blocks are produced very quickly, reducing network stability and making attacks (e.g., **51% attacks**) more feasible.
- **If mining is too hard:**
Mining becomes unprofitable for many participants, causing **miners to leave the network**, which can reduce the overall **hash power and security** of the blockchain.

Q4. UTXO Database (8 marks)

You are given a table that shows the initial state of the UTXO database. Show the final state of the UTXO database after executing all valid transactions listed below. Cross out the UTXOs in the table that have been spent and add new UTXOs in empty spaces. Ignore any invalid transactions.

The following transactions are broadcast to the network in order:

TX5

Inputs:

- (TX1, 0)

Outputs:

- Dave → 2 BTC
- Alice → 3 BTC

TX6

Inputs:

- (TX3, 0)

Outputs:

- John → 6 BTC
- Charlie → 2 BTC

TX7

Inputs:

- (TX2, 1)
- (TX4, 2)

Outputs:

- Eve → 6 BTC
- Bob → 1 BTC

TX8

Inputs:

- (TX1, 0)

Outputs:

- Frank → 5 BTC

Tx ID	Output Index	Owner	Value (BTC)
TX1	0	Alice	5
TX2	1	Bob	3
TX3	0	Charlie	7
TX4	2	Alice	4
TX5	0	Dave	2
TX5	1	Alice	3
TX7	0	Eve	6
TX7	1	Bob	1

Q5. Transaction Fee (15 marks)

Two Bitcoin transactions are currently waiting in the **mempool**.

Transaction X

Transaction size: **300 bytes**

Inputs:

- (TX11, 0) → 0.00003000 BTC
- (TX22, 1) → 0.00001000 BTC

Outputs:

- Alice → 0.00001150 BTC
- Bob → 0.00002100 BTC

Transaction Y

Transaction size: **120 bytes**

Inputs:

- (TX31, 0) → 0.00003800 BTC

Outputs:

- Charlie → 0.00003300 BTC

- a) Compute the transaction fee in satoshis for both transactions (5 marks)
 b) Compute the fee rate in satoshis per byte for both transactions (5 marks)
 c) Which transaction is more likely to be selected first by miners, and why? (5 marks)

a)

Transaction X

Inputs

$$0.00003000 + 0.00001000 = 0.00004000 \text{ BTC}$$

Outputs

$$0.00001150 + 0.00002100 = 0.00003250 \text{ BTC}$$

Fee

$$0.00004000 - 0.00003250 = 0.00000750 \text{ BTC}$$

Convert to satoshis:

$$0.00000750 \times 100,000,000 = 750 \text{ satoshis}$$

Transaction Y

Inputs

$$0.00003800 \text{ BTC}$$

Outputs

$$0.00003300 \text{ BTC}$$

Fee

$$0.00003800 - 0.00003300 = 0.00000500 \text{ BTC}$$

Convert to satoshis:

$$0.00000500 \times 100,000,000 = 500 \text{ satoshis}$$

b)

Transaction X

Fee rate

$$750 / 300 = 2.5 \text{ satoshis per byte}$$

Transaction Y

Fee rate

$$500 / 120 = 4.17 \text{ satoshis per byte}$$

c)

Miner Choice

Even though **Transaction X** has the larger total fee (750 satoshis)

Transaction Y has the larger fee rate (4.17 satoshis per byte)

Miners select **Transaction Y** because it offers a higher **reward per byte of block space**.

Therefore, **Transaction Y** will be selected first.

Q6. Transaction Types (12 marks)

Different Bitcoin transaction types place their **script components and related data** in different parts of the transaction.

Using the options provided below, indicate **which items appear in the Input Block, Output Block, and Witness Block** for each transaction type. Also specify the **transaction version**.

Possible entries:

Legacy

SegWit v0

SegWit v1

scriptSig

scriptPubKey

publicKeyHash

scriptHash

signature(s)

publicKey(s)

multiSigScript

redeemScript

witnessScript

taprootKey

— (for empty or not applicable)

Transaction Type	Version	Input Block	Output Block	Witness Block
P2PKH	Legacy	scriptSig	scriptPubKey	—
P2MS	Legacy	signature(s)	multiSigScript	—
P2SH	Legacy	scriptSig redeemScript	scriptHash	—
P2WPKH	SegWit v0	—	publicKeyHash	signature(s) publicKey(s)
P2WSH	SegWit v0	—	scriptHash	signature(s) witnessScript
P2TR	SegWit v1	—	taprootKey	signature(s)